BLÜK

Tienda online

Memoria del Proyecto

Xavier Morro Sanz · Lidia Vargas

Los documentos, elementos gráficos, vídeos, transparencias y otros recursos didácticos incluidos en este contenido pueden contener imprecisiones técnicas o errores tipográficos. Periódicamente se realizan cambios en el contenido. Fomento Ocupacional FOC SL puede realizar en cualquier momento, sin previo aviso, mejoras y/o cambios en el contenido.

Es responsabilidad del usuario el cumplimiento de todas las leyes de derechos de autor aplicables. Ningún elemento de este contenido, ni parte de este contenido puede ser reproducida, almacenada o introducida en un sistema de recuperación, ni transmitida de ninguna forma ni por ningún medio, sin la previa autorización por escrito de Fomento Ocupacional FOC SL.

Este contenido está protegido por la ley de propiedad intelectual e industrial. Pertenecen a Fomento Ocupacional FOC SL los derechos de autor y los demás derechos de propiedad intelectual e industrial sobre este contenido.

Sin perjuicio de los casos en que la ley aplicable prohíbe la exclusión de la responsabilidad por daños, Fomento Ocupacional FOC SL no se responsabiliza en ningún caso de daños indirectos que se deriven del uso de este contenido.

© 2026 Fomento Ocupacional FOC SL todos los derechos reservados.

Índice general

1 Estudio inicial previo a la realización del proyecto 5
1.1 Clasificación de empresas del sector por organización y servicio . . . 5
1.1.1 Según su modelo de negocio . . . 5
1.1.2 Según su estructura organizativa . . . 5
1.1.3 Según el tipo de producto o servicio . . . 5
1.2 Ejemplo de estructura organizativa del sector . . . 6
1.3 Necesidades demandadas que cubre el proyecto . . . 6
1.4 Descripción del proyecto . . . 7
1.5 Justificación del tipo de proyecto elegido . . . 7
1.6 Características principales del proyecto elegido . . . 8

2 Identificación de necesidades y diseño del proyecto 9
2.1 Estudio inicial y planificación del proyecto . . . 9
2.1.1 Fases del proyecto y su contenido . . . 9
2.1.2 Especificar los objetivos del proyecto . . . 9
2.1.3 Especificar recursos hardware y software . . . 9
2.1.4 Especificar recursos materiales y personales . . . 10
2.1.5 Asociación de fases y recursos . . . 10
2.2 Aspectos fiscales y laborales . . . 10
2.2.1 Obligaciones de protección de datos (RGPD / LOPDGDD) . . . 10
2.2.2 Aspectos fiscales y laborales de la empresa . . . 11
2.3 Viabilidad económica . . . 11
2.3.1 Presupuesto económico estimado (Primer año de explotación) . . . 11
2.3.2 Financiación necesaria y subvenciones . . . 11

3 Modelado de la solución y desarrollo de módulos 12
3.1 Arquitectura general del sistema . . . 12
3.2 Tabla de componentes y tecnologías . . . 12
3.3 Arquitectura física y direccionamiento de red . . . 13
3.3.1 Tabla de direccionamiento IP interno . . . 13
3.4 Diseño de Base de Datos . . . 13
3.4.1 Diagrama Entidad-Relación (ERD) . . . 13
3.4.2 Detalle de tablas relacionales . . . 15
3.4.3 Reglas de integridad aplicadas . . . 15
3.5 Implementación de los módulos de la aplicación . . . 15
3.5.1 1. Autenticación y control de accesos . . . 15
3.5.2 2. Carrito de compras híbrido . . . 16
3.5.3 3. Transacción de pedidos y reserva de stock . . . 16
3.5.4 4. Cancelación de pedidos y reposición automática . . . 17

4 Gestión de riesgos, salvaguardas e incidencias 18
4.1 Riesgos identificados y mitigaciones . . . 18

4.2 Salvaguardas implementadas en la rama de producción 19
4.3 Registro de incidencias técnicas y resoluciones 19
4.3.1 Incidencia 1: Error en build context por carpetas locales en Windows (Docker) . 19
4.3.2 Incidencia 2: Conflicto de nombres de contenedores activos en el host 19
4.3.3 Incidencia 3: Conflictos de permisos de escritura de sesiones entre entornos . . . 20

5 Plan y registro de pruebas de aceptación 21
5.1 Estrategia de pruebas y aseguramiento de calidad 21
5.2 Pruebas automatizadas (PHPUnit) 21
5.2.1 Casos de prueba automatizados cubiertos 21
5.2.2 Comando de ejecución y resultado 22
5.3 Pruebas manuales y registro de resultados 22
5.3.1 Tabla de registro de pruebas manuales 22

6 Desarrollo sostenible y Objetivos de Desarrollo Sostenible (ODS) 25
6.1 ODS relevantes para la actividad profesional 25
6.1.1 ODS 9 — Industria, innovación e infraestructura 25
6.1.2 ODS 12 — Producción y consumo responsables 25
6.1.3 ODS 13 — Acción por el clima 25
6.2 Análisis de riesgos y oportunidades en sostenibilidad 26
6.3 Acciones y compromisos profesionales 26

1 Estudio inicial previo a la realización del proyecto

1.1 Clasificación de empresas del sector por organización y servicio

El proyecto se enmarca dentro del sector del comercio electrónico (e-commerce), concretamente en la venta online de productos textiles y moda urbana. Las empresas de este sector pueden clasificarse bajo diversos criterios organizativos y comerciales:

1.1.1 Según su modelo de negocio

- **B2C (Business to Consumer):** Empresas que venden directamente al consumidor final. BLÜK se clasifica bajo esta categoría, ya que distribuye prendas y accesorios directamente a clientes individuales a través de su portal web.
- **B2B (Business to Business):** Transacciones comerciales entre empresas.
- **C2C (Consumer to Consumer):** Portales de compraventa o intercambio entre consumidores finales.

1.1.2 Según su estructura organizativa

- **Microempresas:** Menos de 10 trabajadores. BLÜK nace como una microempresa de nueva creación, gestionada inicialmente por sus propios socios fundadores (Xavier Morro y Lidia Vargas), quienes asumen las tareas técnicas, de catálogo y de administración.
- **Pequeñas y medianas empresas (PYMEs):** Organizaciones con plantillas mayores que requieren estructuras de delegación formal.
- **Grandes corporaciones:** Cadenas de distribución multinacionales con almacenes inteligentes y distribución global.

1.1.3 Según el tipo de producto o servicio

- **Productos físicos:** Ropa, calzado y accesorios textiles (como el catálogo de ropa urbana de estilo surfer y skater de BLÜK).
- **Productos digitales y servicios:** Descargas de software, consultoría o plataformas SaaS.

1.2 Ejemplo de estructura organizativa del sector

Para una tienda de ropa online urbana en crecimiento, la estructura matricial recomendada incluye los siguientes departamentos funcionales:

- **Dirección General:** Toma de decisiones estratégicas, asignación presupuestaria y alianzas de marca.
- **Departamento de Marketing y Redes Sociales:** Encargado de la publicidad online, gestión de influencers, posicionamiento SEO/SEM y diseño de campañas estacionales.
- **Departamento de Ventas y Atención al Cliente:** Gestión de devoluciones, resolución de dudas del proceso de checkout y soporte postventa.
- **Departamento Técnico (TI):** Desarrollo, mantenimiento correctivo/evolutivo y despliegue de la infraestructura web.
- **Logística y Almacén:** Control del stock físico, empaquetado de pedidos y coordinación con la agencia de transportes.

En la fase inicial de **BLÜK**, todas estas responsabilidades recaen directamente sobre los socios fundadores, simplificando la comunicación pero aumentando la carga polivalente de cada rol.

1.3 Necesidades demandadas que cubre el proyecto

BLÜK cubre un conjunto claro de necesidades tanto comerciales para la marca como operacionales y funcionales para los compradores:

Necesidad del Sector	Implementación en BLÜK
Presencia digital de marca	Creación de un catálogo online accesible desde navegadores web tradicionales y dispositivos móviles.
Venta continua 24/7	Tienda online automatizada que permite agregar productos al carrito y confirmar pedidos a cualquier hora.
Control de inventario en tiempo real	Descuento automático de stock al finalizar una compra y reposición del mismo en caso de cancelación administrativa de un pedido.
Seguridad de información sensible	Cifrado seguro de contraseñas de usuarios registrados y protección contra vulnerabilidades comunes como inyección SQL y ataques CSRF.
Autonomía administrativa	Un panel privado protegido para que los administradores controlen el catálogo y los pedidos globales sin depender de un desarrollador.

1.4 Descripción del proyecto

BLÜK es una tienda online de ropa urbana de estilo surfer y skater. El sistema centraliza la exposición de prendas organizadas por categorías y automatiza de extremo a extremo el flujo transaccional de compra del cliente final y la administración interna de stock y pedidos.

La aplicación consta de dos áreas bien delimitadas: 1. **Frontend del Cliente:** Navegación del catálogo de ropa, visualización de productos por categorías, buscador de productos, carrito persistente híbrido y vista de su historial de compras en su perfil. 2. **Panel de Administración (Backoffice):** Interfaz protegida por middleware para la administración de productos (CRUD con carga de imágenes y desactivación lógica), control de stock físico, listado de usuarios y gestión de pedidos globales (con cancelación y reposición automática de existencias).

1.5 Justificación del tipo de proyecto elegido

El desarrollo de una aplicación web de e-commerce transaccional con arquitectura MVC responde a las siguientes motivaciones profesionales y pedagógicas del ciclo de Desarrollo de Aplicaciones Web (DAW):

- **Integración Multidisciplinar:** Obliga a combinar todas las áreas del ciclo formativo: maquetación semántica y de estilos visuales (CSS/Tailwind), programación lógica del backend y del flujo transaccional (Laravel 11 / PHP), diseño y modelado de bases de datos relacionales (MySQL) y virtualización/despliegue del entorno (Docker).
- **Idoneidad del Framework:** Laravel 11 es el estándar de facto en desarrollo PHP, ofreciendo de forma nativa Eloquent ORM para prevenir inyecciones SQL, andamiajes de autenticación seguros (Breeze) y herramientas de pruebas muy pulidas (PHPUnit).
- **Resolución de Problemas Complejos:** Desafíos reales como evitar la sobreventa de stock bajo concurrencia y la sincronización de carritos anónimos a base de datos.

1.6 Características principales del proyecto elegido

- **Arquitectura MVC limpia:** Organización de código en modelos Eloquent (`app/Models`), vistas de servidor Blade (`resources/views`) y controladores HTTP (`app/Http/Controllers`).
- **Base de datos relacional:** MySQL 8.4 normalizada para garantizar la consistencia en el histórico de facturación (precios de venta congelados en `order_items`).
- **Gestión híbrida de sesiones:** Doble sistema de carrito (sesión local del navegador para invitados y almacenamiento relacional persistente para usuarios autenticados con sincronización post-login).
- **Despliegue y portabilidad:** Estructura contenerizada completa con Docker y Docker Compose para aislar el servidor web, el runtime de PHP y el motor de base de datos MySQL en una red bridge interna dedicada.

2 Identificación de necesidades y diseño del proyecto

2.1 Estudio inicial y planificación del proyecto

2.1.1 Fases del proyecto y su contenido

El desarrollo de la tienda online BLÜK se planificó en cinco fases fundamentales secuenciadas a lo largo de un período estimado de 12 semanas:

- **Fase 1: Análisis y diseño (Semanas 1–2):** Estudio detallado de requisitos funcionales y no funcionales, modelado de base de datos relacional y definición del esquema conceptual.
- **Fase 2: Desarrollo backend (Semanas 3–6):** Instalación del framework Laravel 11, configuración de base de datos en Docker, migración de tablas, andamiaje de autenticación (Breeze) y desarrollo de modelos, controladores e inyección de transacciones atómicas.
- **Fase 3: Desarrollo frontend (Semanas 7–9):** Creación de plantillas Blade dinámicas y maquetación visual responsive con Tailwind CSS, adaptando el catálogo, el carrito y el panel de administración.
- **Fase 4: Integración y pruebas (Semanas 10–11):** Ejecución de pruebas automatizadas unitarias/funcionales con PHPUnit y pruebas de usabilidad manuales en navegador.
- **Fase 5: Despliegue (Semana 12):** Dockerización del entorno productivo mediante contenedores independientes para Nginx, PHP y MySQL.

2.1.2 Especificar los objetivos del proyecto

- **Objetivo 1:** Implementar una tienda online funcional de e-commerce que cubra de manera integral la compra de productos.
- **Objetivo 2:** Diseñar y estructurar una base de datos normalizada para evitar redundancias y garantizar la consistencia transaccional.
- **Objetivo 3:** Aplicar mecanismos de seguridad integrados, como cifrado de contraseñas (bcrypt), validación de formularios y tokens CSRF.
- **Objetivo 4:** Implementar roles diferenciados para el control de accesos (`cliente` y `admin`).
- **Objetivo 5:** Proveer un despliegue contenerizado portable y replicable en cualquier servidor destino.

2.1.3 Especificar recursos hardware y software

- **Recursos Hardware:**
  - Ordenador personal de desarrollo (Procesador equivalente a Intel i5 o AMD Ryzen 5, 8 GB de memoria RAM, almacenamiento SSD).
  - Dispositivos móviles (smartphone) para verificar la adaptabilidad visual de la interfaz.
- **Recursos Software:**

- Sistema operativo: Windows 10/11 (host de desarrollo) y Linux Alpine/Ubuntu (entorno interno de contenedores Docker).
- Entorno de desarrollo: IDE PHPStorm o VS Code.
- Tecnologías core: PHP 8.2, Laravel 11 y MySQL 8.
- Entorno local: Docker Desktop con Laravel Sail.
- Control de versiones: Git y repositorio remoto en GitHub.
- Navegadores web: Google Chrome, Mozilla Firefox y Microsoft Edge.

2.1.4 Especificar recursos materiales y personales

- **Recursos Personales:** El desarrollo es llevado a cabo de forma íntegra por el alumno Xavier Morro Sanz (con la colaboración de Lidia Vargas para el diseño de catálogo e identidad gráfica), con el soporte metodológico del equipo docente y tutor de FOC.
- **Recursos Materiales:** Equipo informático de desarrollo, conexión a internet de banda ancha y entorno virtual local de base de datos y servidor de correo ficticio de desarrollo.

2.1.5 Asociación de fases y recursos

- **Fase de Análisis y Diseño:** Requiere procesador de textos para documentación técnica y herramientas de modelado de base de datos.
- **Fases de Desarrollo (Backend y Frontend):** Demanda el uso del IDE, motor de base de datos MySQL local y el runtime de PHP montado en el contenedor de Docker Sail.
- **Fase de Pruebas:** Utiliza suites de testing automáticas (PHPUnit) y navegadores web para validación manual de responsive.
- **Fase de Despliegue:** Requiere el orquestador Docker Compose para empaquetar la infraestructura en producción.

2.2 Aspectos fiscales y laborales

2.2.1 Obligaciones de protección de datos (RGPD / LOPDGDD)

Al procesar información de personas físicas (nombre, correo electrónico, contraseñas y registros de compras), el sistema BLÜK cumple con el Reglamento General de Protección de Datos (RGPD) y la Ley Orgánica de Protección de Datos y Garantía de los Derechos Digitales (LOPDGDD): * **Principio de minimización:** Solo se recaban los datos esenciales para procesar el pedido e iniciar sesión. * **Seguridad:** Almacenamiento seguro de contraseñas mediante hash irreversible (bcrypt). * **Derechos ARSOPL:** Los usuarios pueden ejercer sus derechos de acceso, rectificación y supresión directamente desde su perfil personal de cliente.

2.2.2 Aspectos fiscales y laborales de la empresa

Para la explotación comercial real de la marca de ropa urbana: * **Obligaciones fiscales:** Alta en el Censo de Empresarios (modelos 036/037), encuadramiento en el Impuesto de Actividades Económicas (IAE), y liquidaciones periódicas de IVA (modelo 303) e IRPF o Impuesto sobre Sociedades. * **Obligaciones laborales:** Alta de autónomo para los socios fundadores, registro de contratos de personal (si aplica) y cotizaciones sociales correspondientes. * **Prevención de riesgos laborales:** Evaluación ergonómica del puesto de desarrollo de software para mitigar fatiga visual y lesiones musculares.

2.3 Viabilidad económica

2.3.1 Presupuesto económico estimado (Primer año de explotación)

Concepto	Coste Estimado	Función
Dominio web (`.com`)	15 € / año	Nombre identificador de la marca en internet
Hosting VPS básico	120 € / año	Servidor privado virtual para despliegue Docker
Certificado SSL/TLS	0 € (Let's Encrypt)	Encriptación de datos en tránsito (HTTPS)
Servicios de correo	30 € / año	SMTP para notificaciones de pedidos
Imagen de marca inicial	100 €	Logotipo, diseño de camisetas y pegatinas
Contingencias técnicas	50 €	Margen para gastos imprevistos de software
Total Estimado Primer Año	**315 €**	Coste de entrada reducido y asumible

2.3.2 Financiación necesaria y subvenciones

- **Financiación:** Al no requerir infraestructura física ni maquinaria costosa, la inversión de 315 € es asumida al 100% por los propios socios fundadores con fondos personales.
- **Ayudas futuras:** Subvenciones autonómicas para jóvenes emprendedores o bonificaciones del Kit Digital para la implantación de soluciones e-commerce en microempresas.

3 Modelado de la solución y desarrollo de módulos

3.1 Arquitectura general del sistema

La tienda online **BLÜK** se ha desarrollado adoptando el patrón arquitectónico de diseño de software **Modelo-Vista-Controlador (MVC)**, estructurado nativamente por el framework Laravel 11. Este patrón organiza el código separando la interfaz de usuario de la lógica de negocio y persistencia:

- **Modelos (Model):** Representan la lógica de datos y persisten la información haciendo uso de Eloquent ORM. Están ubicados en el directorio `app/Models/`.
- **Vistas (View):** Definen la interfaz gráfica que ve el usuario, utilizando el motor de plantillas Blade de Laravel. Se ubican en `resources/views/`.
- **Controladores (Controller):** Actúan como intermediarios procesando las peticiones HTTP (`Requests`), ejecutando la lógica y retornando las respuestas (`Responses`) o vistas correspondientes. Se ubican en `app/Http/Controllers/`.

3.2 Tabla de componentes y tecnologías

El stack real que conforma la solución de software se resume en la siguiente tabla:

Componente	Tecnología	Función Principal
Frontend	Blade + Tailwind CSS	Interfaz gráfica interactiva y responsive para el usuario.
Backend	Laravel 11 (PHP 8.2)	Control de flujo de peticiones, seguridad y lógica transaccional.
Persistencia	MySQL 8.4	Almacenamiento relacional de usuarios, catálogo, carritos y pedidos.
Servidor Web	Nginx	Servidor HTTP de producción para atender peticiones FastCGI.
Contenedores	Docker & Docker Compose	Virtualización y aislamiento de servicios en redes bridge estáticas.
Control de versiones	Git / GitHub	Gestión del código fuente y control de cambios del equipo.

3.3 Arquitectura física y direccionamiento de red

En producción, el sistema BLÜK se ejecuta en tres contenedores de Docker conectados mediante una red virtual privada bridge interna llamada `bluk_net` con la subred `172.18.0.0/16`.

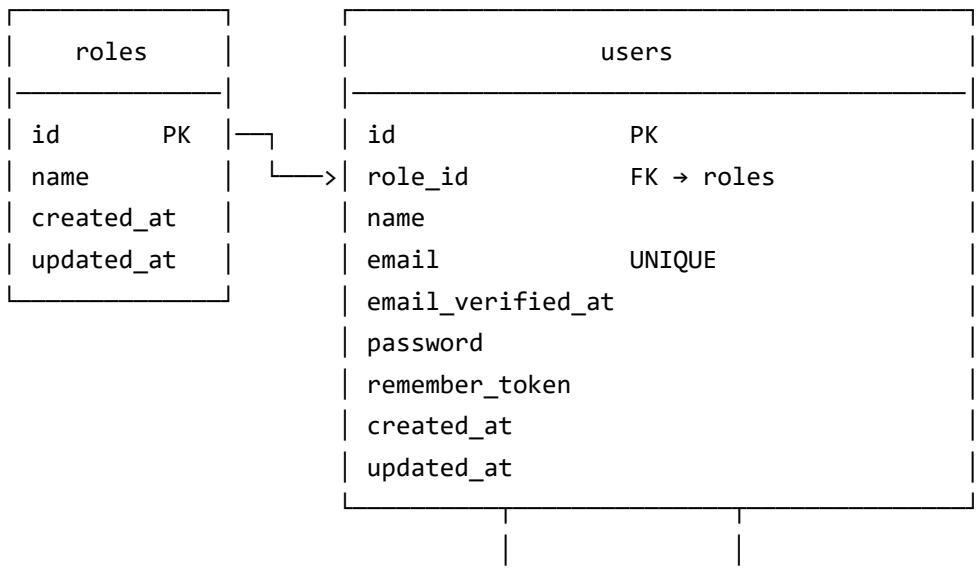
3.3.1 Tabla de direccionamiento IP interno

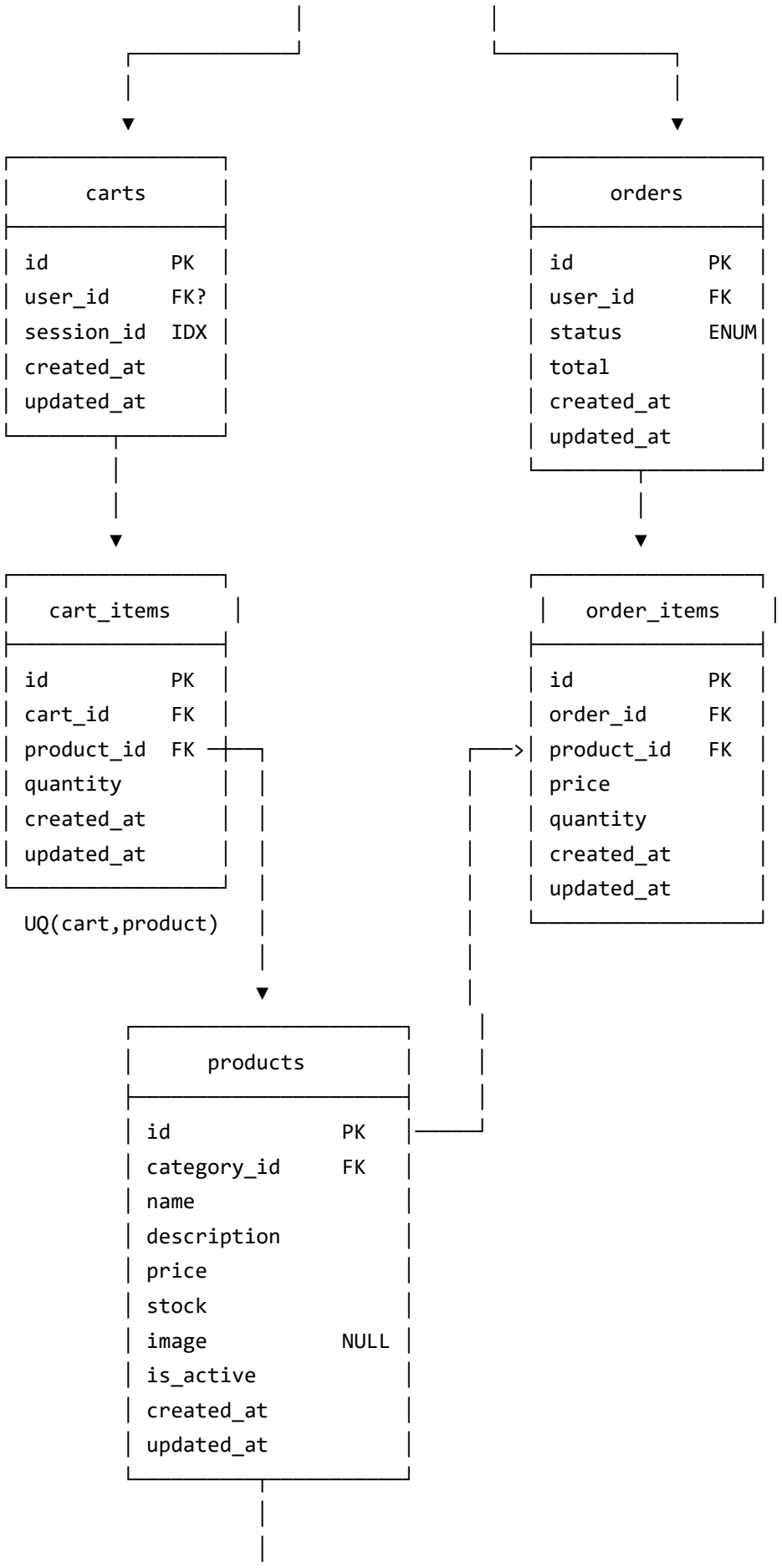
Servicio / Contenedor	Red Docker	IP Interna	Puerto Interno	Puerto Host Mapeado
Nginx (`bluk_prod_ngi`	`bluk_net`	`172.18.0.2`	80	8080 (HTTP)
Laravel App (`bluk_prod_app`	`bluk_net`	`172.18.0.3`	9000	-
MySQL (`bluk_prod_mys`	`bluk_net`	`172.18.0.4`	3306	3306 (Localhost)

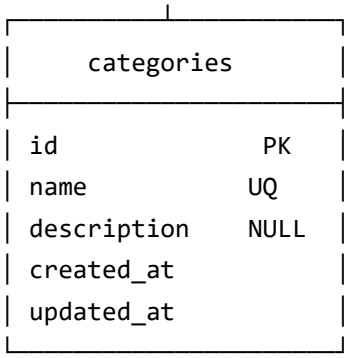
3.4 Diseño de Base de Datos

3.4.1 Diagrama Entidad-Relación (ERD)

La base de datos relacional de la aplicación organiza la información de la tienda siguiendo este esquema estructurado:




carts
id PK
user_id FK?
session_id IDX
created_at
updated_at
orders
id PK
user_id FK
status ENUM
total
created_at
updated_at
cart_items
id PK
cart_id FK
product_id FK
quantity
created_at
updated_at
UQ(cart,product)
order_items
id PK
order_id FK
product_id FK
price
quantity
created_at
updated_at
products
id PK
category_id FK
name
description
price
stock
image NULL
is_active
created_at
updated_at



3.4.2 Detalle de tablas relacionales

- **`roles`:** Define perfiles de acceso autorizados en el sistema (`admin` y `cliente`).
- **`users`:** Cuentas de usuario registradas conectadas a un `role_id` (clave foránea).
- **`categories`:** Organiza los artículos en categorías lógicas de streetwear (Sudaderas, Camisetas, Pantalones, Accesorios).
- **`products`:** Registro del catálogo de artículos físicos con su precio, stock actual en almacén y el estado lógico `is_active`.
- **`carts` y `cart_items`:** Mecanismo relacional de persistencia de productos añadidos al carro. Soporta almacenamiento para visitantes anónimos (`session_id`) y usuarios logueados (`user_id`).
- **`orders` y `order_items`:** Registro consolidado de compras. `order_items` congela el campo `price` al momento de la venta para independizar el pedido histórico de futuras modificaciones en el catálogo.

3.4.3 Reglas de integridad aplicadas

- **`CASCADE` en `categories` y `carts`:** Si se da de baja una categoría o un carrito, los registros hijos asociados se borran automáticamente.
- **`RESTRICT` en `users` y `products` respecto a compras:** Evita borrar usuarios con pedidos completados o productos que aparecen en el histórico de compras de `order_items`.
- **Unique Constraint en `cart_items`:** La clave única compuesta `(cart_id, product_id)` garantiza que los productos agregados de forma duplicada al carrito incrementen su cantidad en lugar de duplicar filas.

3.5 Implementación de los módulos de la aplicación

3.5.1 1. Autenticación y control de accesos

Se utiliza **Laravel Breeze** adaptado para redirecciones lógicas en función del rol asignado. Para proteger el backoffice administrativo de accesos fraudulentos de usuarios clientes o visitantes, se implementó el siguiente middleware interceptor (`AdminMiddleware`):

```php
public function handle(Request $request, Closure $next)
{
    if (auth()->check() && auth()->user()->role->name === 'admin') {
        return $next($request);
    }
    abort(403, 'Acceso no autorizado.');
}
```

3.5.2 2. Carrito de compras híbrido

El flujo gestionado por `CartController` permite: * **Modo Invitado:** Los productos agregados se almacenan como un array indexado dentro de la sesión de PHP en el servidor. * **Modo Autenticado:** Los artículos se persisten directamente en la base de datos MySQL en las tablas `carts` y `cart_items`. * **Fusión de carrito (Post-Login):** Al autenticarse un cliente, un escuchador de eventos de sesión recupera los productos del carrito de sesión temporal de invitado y los fusiona en las tablas correspondientes vinculándolo a su `user_id`.

3.5.3 3. Transacción de pedidos y reserva de stock

La creación de un pedido (`OrderController@store`) realiza un descuento físico del inventario disponible en almacén. Para prevenir inconsistencias y condiciones de carrera bajo solicitudes de compra concurrentes, el proceso completo se encapsula en una transacción de base de datos (`DB::transaction`):

```php
DB::transaction(function () use ($cart, $user) {
    $order = Order::create([
        'user_id' => $user->id,
        'status' => 'pendiente',
        'total' => $cart->total(),
    ]);

    foreach ($cart->items as $item) {
        $product = $item->product;
        if ($product->stock < $item->quantity) {
            throw new Exception("Stock insuficiente para: " . $product->name);
        }
        $product->decrement('stock', $item->quantity);
        $order->items()->create([
            'product_id' => $product->id,
            'price' => $product->price,
            'quantity' => $item->quantity,
        ]);
    }
    $cart->delete();
});
```

3.5.4 4. Cancelación de pedidos y reposición automática

Desde el panel administrativo, cambiar el estado de un pedido a `cancelado` desencadena un evento automático para reabastecer el almacén sumando la cantidad cancelada al stock actual del producto. Este proceso cuenta con guardas lógicas de estado para evitar la reposición duplicada ante llamadas repetidas.

4 Gestión de riesgos, salvaguardas e incidencias

4.1 Riesgos identificados y mitigaciones

Durante la planificación y ejecución de la tienda online **BLÜK**, se identificaron riesgos críticos de integración y operación:

ID	Riesgo Identificado	Severidad	Mitigación Aplicada
R1	Inconsistencia de stock por concurrencia	Alta	Encapsulado del flujo de reserva de stock en transacciones atómicas de base de datos (`DB::transaction`).
R2	Intrusiones fraudulentas al panel administrativo	Muy Alta	Implementación de un middleware exclusivo (`AdminMiddleware`) que intercepta peticiones HTTP y restringe el acceso a administradores.
R3	Pérdida de carritos de visitantes invitados	Media	Persistencia híbrida mediante almacenamiento local en sesión y fusión automática post-login.
R4	Conflictos de dependencias entre entornos locales	Media	Aislamiento completo del entorno de desarrollo y ejecución mediante contenedores virtuales de Docker y orquestación con Docker Compose.

4.2 Salvaguardas implementadas en la rama de producción

Para asegurar que el código en la rama principal (`main`) sea estable, se aplican las siguientes reglas y salvaguardas de desarrollo: * **Prohibición de pushes directos:** Los desarrolladores deben abrir ramas de características y proponer cambios a través de Pull Requests (PR). * **Revisión obligatoria de pares:** Ningún cambio puede ser integrado a la rama principal sin la aprobación manual de los revisores autorizados. * **Pruebas previas automatizadas:** Integración de comandos de verificación antes del merge para asegurar la no regresión del código.

4.3 Registro de incidencias técnicas y resoluciones

A continuación se detallan las tres incidencias técnicas críticas surgidas durante el desarrollo del proyecto y la forma profesional en que se resolvieron:

4.3.1 Incidencia 1: Error en build context por carpetas locales en Windows (Docker)

- **Descripción del problema:** Al intentar construir las imágenes Docker productivas en entornos del sistema operativo Windows, el motor Docker fallaba arrojando un error de archivo no encontrado o solicitud inválida (`invalid file request`) en carpetas internas del directorio `node_modules` y `vendor`. Esto se debe a la imposibilidad de Docker para procesar o seguir enlaces simbólicos creados nativamente por el gestor de paquetes de Windows.
- **Solución técnica:** Se creó un archivo de exclusión `.dockerignore` en la raíz del proyecto para omitir la transferencia de las carpetas de dependencias locales pesadas (`node_modules`, `vendor`) al daemon de Docker. Esto forzó a que cada contenedor instale sus dependencias de forma interna y aislada, acelerando además la velocidad de compilación notablemente.

4.3.2 Incidencia 2: Conflicto de nombres de contenedores activos en el host

- **Descripción del problema:** Al intentar levantar el entorno contenerizado de producción tras haber estado ejecutando el entorno de desarrollo local, el comando fallaba con excepciones de conflicto de nombres, indicando que alias como `bluk_app` o `bluk_mysql` ya se encontraban registrados o en uso por ejecuciones pasadas del proyecto.
- **Solución técnica:** Modificamos el archivo de orquestación de producción (`docker-compose.prod.yml`) inyectándole un nombre de proyecto unívoco usando la directiva `name: bluk_prod`. Además, renombramos de forma única cada contenedor interno del entorno productivo (`bluk_prod_app`, `bluk_prod_nginx` y `bluk_prod_mysql`), evitando interferencias.

4.3.3 Incidencia 3: Conflictos de permisos de escritura de sesiones entre entornos

- **Descripción del problema:** Las ejecuciones en local arrojaban excepciones PHP de denegación de permisos (`Permission denied` en carpetas como `storage/framework/sessions`) al intentar guardar carritos o sesiones de usuario. Esto se produjo porque los archivos temporales creados por el contenedor productivo bajo la cuenta `www-data` no podían ser modificados o escritos por el usuario Sail (`sail`) del contenedor de desarrollo local sobre el mismo volumen compartido en el host.
- **Solución técnica:** Se ejecutó un comando de asignación recursiva de permisos de lectura y escritura como superusuario (`chmod -R 777`) sobre las carpetas `storage` y `bootstrap/cache`, solucionando de manera definitiva la colisión de permisos entre los usuarios de los entornos.

5 Plan y registro de pruebas de aceptación

5.1 Estrategia de pruebas y aseguramiento de calidad

El plan de pruebas del e-commerce **BLÜK** se diseñó con el fin de verificar que todas las funcionalidades y reglas de negocio respondan de manera correcta ante diversos escenarios de operación, asegurando tanto la experiencia de usuario en el frontend como la estabilidad de la lógica de negocio y persistencia en el backend.

El plan de verificación implementado consta de dos niveles independientes: 1. **Pruebas Automatizadas (PHPUnit):** Ejecución de tests en el backend para validar el comportamiento del código ante peticiones controladas, cubriendo redirecciones, middleware y flujos lógicos. 2. **Pruebas Manuales (Aceptación en Navegador):** Inspección visual directa y flujos de punta a punta realizados por el equipo de desarrollo sobre navegadores web para validar responsive e interfaces.

5.2 Pruebas automatizadas (PHPUnit)

Las pruebas automatizadas de backend se ejecutan sobre la base del framework de pruebas PHPUnit incorporado en Laravel 11. Se ubicaron en el directorio de pruebas de integración y características `tests/Feature/`.

5.2.1 Casos de prueba automatizados cubiertos

- **Seguridad y control de acceso:** Validación del middleware administrativo (`AdminMiddleware`), asegurando que las peticiones a rutas privadas `/admin/*` devuelvan código de denegación `403` o redirecciones a `/login` si no se dispone del rol `admin`.
- **Catálogo de productos:** Validación de la carga de la tienda pública, filtrado por categorías, búsqueda de artículos y respuesta con código `404` para productos inactivos.
- **Carrito de compras híbrido:** Adición de productos a la sesión, sincronización automática al base de datos tras la autenticación y vaciado del carro.
- **Checkout y transacciones:** Proceso de creación del pedido, descuento automático de unidades de stock del catálogo físico e interrupción con excepción controlada en caso de intentar comprar productos con inventario insuficiente.
- **Cancelación y reabastecimiento:** Validación de que al cancelar administrativamente un pedido, el stock retorne automáticamente a los productos asociados, controlando ejecuciones sucesivas redundantes.

5.2.2 Comando de ejecución y resultado

La suite de tests completa se ejecuta en el contenedor de desarrollo del proyecto mediante la siguiente instrucción:

```
docker compose exec -u sail laravel.test php artisan test
```

El resultado obtenido es: * **Tests pasados con éxito: 49 tests pasaron de manera exitosa** (128 aserciones ejecutadas con 0 errores y 0 fallos), garantizando la no regresión del código.

5.3 Pruebas manuales y registro de resultados

Para comprobar el diseño visual adaptable (responsive) y los flujos completos desde la perspectiva del comprador final y del administrador, se ejecutaron 12 casos de prueba (CP) en navegadores web.

5.3.1 Tabla de registro de pruebas manuales

ID	Caso de Prueba	Precondición	Acción Realizada	Resultado Esperado	Estado
CP-01	Registro de nuevo cliente	Visitante no registrado	Rellenar formulario de registro con datos correctos.	Cuenta creada e inicio de sesión automático redirigiendo al catálogo.	**Cumple**
CP-02	Intrusión a rutas adminis-trativas	Sesión iniciada con rol `cliente`	Intentar acceder directamente a `http://locall`	Retorno de código de error de acceso restringido `403` (Forbidden).	**Cumple**
CP-03	Filtrado del catálogo	Catálogo con productos cargados	Pulsar en la categoría “Sudaderas” en la barra lateral.	Visualización exclusiva de sudaderas en el escaparate.	**Cumple**

ID	Caso de Prueba	Precondición	Acción Realizada	Resultado Esperado	Estado
CP-04	Búsqueda por palabra clave	Catálogo activo	Buscar la palabra “Oversize” en el cuadro de búsqueda.	Filtrado del catálogo listando solo prendas con esa palabra clave.	**Cumple**
CP-05	Carrito persistente invitado	Visitante no registrado	Añadir dos camisetas al carrito y cerrar el navegador web.	Al reabrir la web, los artículos continúan guardados en el carrito de sesión.	**Cumple**
CP-06	Sincronización al loguearse	Carrito de invitado con artículos	Iniciar sesión con una cuenta de cliente existente.	Los artículos del carrito de sesión se migran a la base de datos bajo el perfil.	**Cumple**
CP-07	Proceso de compra completo	Cliente logueado con stock disponible	Confirmar y finalizar pedido desde el carrito de compras.	Pedido registrado como “pendiente” y stock decrementado en tiempo real.	**Cumple**
CP-08	Compra con stock excedido	Cliente logueado en checkout	Intentar forzar la compra de una prenda en cantidad superior al stock.	Bloqueo del proceso con mensaje flash de aviso de stock insuficiente.	**Cumple**
CP-09	CRUD de productos admin	Administrador autenticado	Crear un producto con foto y modificar su precio desde el panel.	Producto creado correctamente con imagen almacenada en disco.	**Cumple**

ID	Caso de Prueba	Precondición	Acción Realizada	Resultado Esperado	Estado
CP-10	Ocultación por desactivación	Administrador autenticado	Desactivar un producto marcando su opción `is_active = false`.	El producto deja de mostrarse en el catálogo del cliente inmediata-mente.	**Cumple**
CP-11	Cancelación administrativa	Administrador autenticado	Modificar el estado de un pedido pendiente a “cancelado”.	El stock de las prendas asociadas al pedido se restituye auto-máticamente.	**Cumple**
CP-12	Adaptabilidad del diseño	Diversos perfiles de usuario	Redimensionar navegador y probar la web desde dispositivos móviles.	Menú móvil colapsable y rejilla de productos reajustada co-rrectamente.	**Cumple**

Todos los casos de prueba manuales fueron ejecutados de manera exitosa, confirmando el correcto funcionamiento visual, funcional y técnico de la aplicación.

6 Desarrollo sostenible y Objetivos de Desarrollo Sostenible (ODS)

6.1 ODS relevantes para la actividad profesional

En el marco del desarrollo de la plataforma de comercio electrónico **BLÜK**, se han seleccionado tres Objetivos de Desarrollo Sostenible (ODS) que se vinculan de manera directa con la actividad profesional del desarrollo web y la digitalización comercial:

6.1.1 ODS 9 — Industria, innovación e infraestructura

La creación de plataformas de software sólidas, estables y contenerizadas impulsa la digitalización de la economía local. BLÜK aporta valor directo al tomar una marca de ropa urbana de carácter físico y dotarla de una infraestructura digital robusta que automatiza procesos y facilita la innovación comercial. * **Relación con BLÜK:** La modularidad de la arquitectura MVC de Laravel y el empaquetado mediante contenedores de Docker garantizan que la infraestructura de la tienda pueda escalar u hospedarse de forma portable en cualquier servidor en la nube sin requerir la reconstrucción de sus piezas técnicas.

6.1.2 ODS 12 — Producción y consumo responsables

La digitalización comercial ayuda a optimizar las cadenas de suministro. El control preciso del inventario y la venta directa reducen intermediarios, disminuyen el desperdicio de existencias y promueven una comercialización consciente de recursos textiles en el sector de la moda. * **Relación con BLÜK:** El sistema de control de stock en tiempo real y la reposición automática de productos ante pedidos cancelados previenen descuadres de inventario. Esto permite a los gestores de la marca fabricar o almacenar solo lo necesario, evitando el sobrestock y fomentando un modelo de producción “just in time”. Además, la maquetación ligera mediante Tailwind CSS optimiza el tráfico de datos transferidos, reduciendo el consumo energético de procesamiento en los dispositivos de los clientes.

6.1.3 ODS 13 — Acción por el clima

El sector tecnológico genera impacto energético en servidores y centros de datos. La optimización del código del lado del servidor (PHP) y de las consultas a base de datos disminuye el tiempo de CPU requerido, reduciendo directamente las emisiones indirectas de carbono del hospedaje web. * **Relación con BLÜK:** BLÜK elimina por completo la necesidad de facturas y registros físicos en papel, centralizando todo el historial transaccional de manera digital en la base de datos relacional. Asimismo, consolidar las ventas a través de una plataforma online optimiza las rutas de envío y logística en comparación con los desplazamientos físicos individuales de los clientes a puntos de venta locales.

6.2 Análisis de riesgos y oportunidades en sostenibilidad

ODS	Oportunidades Profesionales	Riesgos Asociados
ODS 9	Fomentar la adopción de arquitecturas portables y libres (Docker, MySQL, PHP) en pequeñas y medianas empresas.	Obsolescencia rápida de herramientas o alta dependencia de plataformas en la nube de carácter privativo.
ODS 12	Optimizar interfaces web para reducir la transferencia de datos y la huella de procesamiento del cliente.	Fomentar compras impulsivas e insostenibles si el proceso de checkout se simplifica en exceso sin consciencia de consumo.
ODS 13	Reducir el consumo en el servidor depurando consultas N+1 y aplicando índices eficientes en la base de datos.	Aumento desmedido de la huella de carbono digital por el uso de servidores VPS ineficientes o no optimizados.

6.3 Acciones y compromisos profesionales

- **Eficiencia en código:** Optimizar las relaciones y consultas Eloquent para evitar cargas de datos innecesarias a memoria, reduciendo el consumo del CPU del servidor web.
- **Políticas "Paperless":** Eliminar la impresión física de tickets y hojas de pedidos sustituyéndola por una vista digital interactiva del historial en la base de datos del perfil del comprador.
- **Software libre:** Priorizar el uso de tecnologías de código abierto (open-source) que reduzcan barreras económicas y fomenten la interoperabilidad técnica y la transparencia.